

Python Interview Preparation

Written by: Muhammad Nouman

1. What is python ?

Answer : Python ek high-level, interpreted, object-oriented programming language hai. Ye simple syntax ki wajah se famous hai aur web development, AI, automation, scripting aur data science me use hoti hai.

Features:

- Easy syntax
- Readable code
- Large community support
- Cross-platform
- Huge libraries support

Example:

```
print("Hello World")
```

2. Python Interpreted Language q Kehlati Hai?

Answer: Python interpreted language hai kyun ke iska code runtime par Python Virtual Machine (PVM) execute karti hai, c/c++ ki tarha pehle pure code ko machine code me convert nahi kiya jata.

3. python interpreted hai ya compiled?

Answer: python interpreted b hai or compiled b pehle step may code bytecode may Compile hota hai phir PVM (Python Virtual Machine) us bytecode ko line by line execute krti hai

4. What are Data Types in Python?

Answer: Python me **Data Types** different types ka data store karne ke liye use hote hain. Har variable ka ek data type hota hai jo batata hai ke us variable me kis type ka data store hoga.

Types of Data Types in Python

Category	Data Types	Example
Numeric	int, float, complex	10, 4.12, 2+3j
Sequence	list, tuple, range	[1,2], (1,2),
Text	String	"Hello"
Set	Set	{1,2,3}
Mapping	Dict	{"a" : 1}
Boolean	bool	True, False
Binary	bytes, byte array, memoryview	b"hello"

Python may built-in data types wo hote hain jo different types ka data store krte hain.

5. Variables kia hote hain?

Answer: Variable aik named memory location hota hai jo data ko temporarily store karne ke liye use hota hai.

6. What is mutable and immutable in python?

Answer:

i. mutable wo data types hote hain jin ki value ko change kia ja skta hai Example:

- List
- Dict
- Set

ii. immutable data types wo hote hain jin ki value ko change nahi kia ja skta Example:

- Tuple
- String
- Int

7. What is the difference between list and tuple?

Answer:

i. List mutable hai isko square brackets may define kia jata hai or ye tuple se thori slow hoti hain

ii. tuple immutable hai or isko parentheses may define kia jata hai tuple list se thori fast hoti hai

8. What is type casting in python?

Answer: aik data type ko dusre data type may convert krna type casting kehlata hai python may do tarha ki type casting hoti hai

i. Implicit

Python automatically convert krta hai jb zarorat ho

ii. Explicit

Hum manually conversion krte hain

9. Operators in python ?

Answer:

I. Arithmetic ii. Comparison iii. Logical Iv. Assignment v. Membership vi. Identity vii. Bitwise

i. Arithmetic operators

Oprataor	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Division

%	Modulus (Remainder)
**	Exponent (Power)

ii. Comparison Operators

Operator	Meaning
==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal

iii. Logical Operators

Operator	Meaning
And	Dono Conditions True hon
Or	Koi Aik Condition True ho
Not	Result ko reverse krta hai

Iv. python may kitni Type k Operators hote hain?

- Arithmetic
- Comparison
- Logical
- Assignment
- Membership
- Identity
- Bitwise

10. What is a function?

Answer: function reusable block of code hota hai jo specific task ko perform krta hai

Example:

```
def greet(name):
    return f"hello{name}"
```

11. What is the difference between parameter and arguments:

Answer: Parameters wo variables hote hain jo function define karte waqt parentheses () ke andar likhe jate hain.

Example:

```
def greet(name):
    print(name)
```

name aik parameter hai

Arguments wo actual values hoti hain jo function ko call karte waqt Pass ki jati hain.

Types of Arguments

- Positional Arguments
- Keyword arguments
- Default arguments
- variable-length Arguments

Examples:

i. Positional Arguments

Defination: jaha order tarteeb matter krta ho

Examples:

i.

```
def student(name, math, science, english):
    print(f"{name} got {math}, {science}, {english} marks")
student("Nouman", 80, 75, 90)
```

Output: Nouman got 80, 75, 90 marks

ii.

```
def book(title, author, pages):
    print(f"{title} by {author} has {pages} pages")
book('Python', 'John', 300)
```

Output: Python by John has 300 pages

iii.

```
def rectangle(length, width):
    area = length * width
    print(f"Area is: {area}")
rectangle(10, 5)
```

Output: Area is: 50

iv.

```
def student(name, age, city):
    print(f"{name} is {age} years old from {city}")
student("Ali", 20, "Karachi")
```

Output: Ali is 20 years old from Karachi

v.

```
def order(item, quantity, price):
    print(f"{item} x{quantity} costs {price}")
order('Burger', 2, 600)
```

Output: Burger x2 costs 600

ii. Keyword Arguments

Defination: Keyword arguments wo hote hain jin mein function call karte waqt parameter ka naam likh kar value pass ki jati hai

Examples:

i.

```
def student(name, age, city):
    print(f"{name} is {age} years old from {city}")
```

```
student("Ahmed", city="Islamabad", age=22)
```

Output: Ahmed is 22 years old from Islamabad

ii.

```
def employee(name, department, salary):  
    print(name, department, salary)  
employee(name="Nouman", salary=50000,  
department="Farooqabad")
```

Output: Nouman 50000 Farooqabad

iii.

```
def add(a, b, c):  
    print(a, b, c)  
add(c=30, a=10, b=20)
```

Output: 10, 20, 30

iv.

```
def info(name, country, profession):  
    print(f"{name} {country} {profession}")  
info(profession="Engineer", name="Nouman",  
country="Pakistan")
```

Output: Nouman pakistan Engineer

iii. Default Arguments

Defination: jaha parameter ki value pehle se set ho

Examples:

i.

```
def greet(name, message="Hello"):  
    print(f"{message}, {name}!")  
greet("Ali")
```

Output: Hello, Ali!

ii.

```
def student(name, age=18, city="Karachi"):  
    print(f"{name} is {age} years old from {city}")
```

student("Ahmed") output: age=18, city=Karachi

student("Ahmed", 20) output: city=Karachi

student("Ahmed", 20, "Lahore") output: sab custom

Iv. Variable-Length Arguments

Answer: Jab hume nahi pata hota k function may kitni value ay gi to hum variable-length arguments use karte hain python may inki do types hoti hain

***A) argos (Positional Variable-length)**

Definition: jahan jaha multiple values bina key k pass krni ho

Examples:

i.

```
def total(*numbers):  
    total = 0  
    for n in numbers:  
        total += n
```

```
print(total)
```

```
total(10, 20, 30, 40)
```

Output: 100

ii.

```
def names(*students):  
    print(" ".join(students))  
names("Ali", "Nouman", "Ahmad")  
Output: Ali Nouman Ahmad
```

****B) kwargs (keyword variable-length)**

Definition: jahan multiple key-value pairs pass krni ho

Examples:

```
def print_details(**kwargs):  
    print(f"Data received: {kwargs}")  
    for key, value in kwargs.items():  
        print(f"{key} = {value}")
```

```
print_details(name="Ali", age=20, city="Karachi",  
profession="Developer") Output:
```

name = Ali

age = 20

city = Karachi

profession = Developer

12. What is the lambda function?

Answer : Lambda function aik anonymous (bina naam ka) function hota hai jo ek hi expression rakhta hai. Ye chote aur simple operations ke liye use hota hai.

Example:

```
square = lambda x: x **2  
print(square(5)) output 25
```

i. map() kia hota hai

map() aik built-in function hai jo data ko change krne k lia use hota hai **Example:**

```
numbers = [1, 2, 3, 4]  
result = map(lambda x: x * 2, numbers)  
print(list(result))
```

Output: [2, 4, 6, 8]

ii. Filter() kia hota hai ?

filter() aik built-in function hai jo selected data ko return krta hai

Example:

```
numbers = [1, 2, 3, 4, 5, 6]  
result = filter(lambda x: x % 2 == 0, numbers)  
print(list(result))
```

Output: [2, 4, 6]

iii. What is reduce()?

reduce() iterable ki tamam values ko combine karke ek single value return karta hai.

12. what is *args and **kwarg?

Answer:

i. *args

multiple Positional arguments store karta hai

ii. **kwargs

Multiple keyword arguments receive karta hai

13. What is scope?

Answer: Scope variable ki **accessibility (visibility)** define karti hai, yani variable program ke kis hisse me access kiya ja sakta hai.

Python me **2 main types** ki scope hoti hain.

i. **Local Scope** wo variable hota hai jo function ke andar define hota hai aur sirf usi function ke andar access kiya ja sakta hai.

ii. **Global Scope** variable jo function k bahir define hota hai, puri file may accessible hota hai

Important point: agar function k andar global variable ko modify karna hoto 'global' keyword use hota hai warna python usay local variable assume kr leta hai.

Examples: Local Scope

i. def func():

 x = 50

 print(x)

func() output: 50

ii. **Global Scope**

 x = 100

def func():

 print(x)

func() output: 100

iii. **Global Keyword (Important)**

 x = 100

def func():

 global x

 x = 200

 print(x)

func() output: 200

Common interview trap question

my_var = 10

def test():

 print(my_var) Line 1

```
my_var = 20 Line 2
print(my_var) Line 3
```

test()

Question: output kia hoga

Answer: unboundlocalError

Reason: python function k andar my_var = 20 dekh kr usay local variable treat krta hai line 1 pe print krte waqt local variable exist nahi krta is lia error ata hai.

Solution:

```
my_var = 10
def test():
    global my_var Add this line
    print(my_var) output: 10
    my_var = 20 output: modifies global
    print(my_var) output: 20
test()
```

14. What is recursion?

Answer: Recursion wo technique hai jisme ek function khud ko call karta hai kisi problem ko chhote parts me solve karne ke liye.

Recursion ke 2 important parts hote hain.

- Base case: (rukne ki condition)
- Recursive case (khud ko call krne ki condition)

Example:

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)
```

15. What is exception handling

Answer: Exception Handling aik mechanism hai jo program ko runtime errors ki wajah se crash hone se bachata hai.

Keywords:

- **try:** Jis code mein error aa sakta ho usay try block mein likhte hain.
- **except:** Error aane par except block execute hota hai.
- **finally:** finally block hamesha execute hota hai, chahe error aaye ya na aaye.
- **raise:** Khud se exception generate karne ke liye use hota hai.

Yaad rakhne k lia

- try = risky code
- except = error handle karo
- finally = hamesha chalega
- raise = khud error generate karo

16. Difference between == and is?

Answer: i. == do objects ki value compare karta hai jabke

ii. is do objects ki memory location compare krta hai

Example:

```
a = [1,2]
b = [1,2]

print(a == b)
print(a is b)
```

17. Deep copy vs shallow copy

Answer: i. deep copy

Deep Copy poore object aur uske nested objects ki bhi alag copy banati hai. Agar original me changes karo to copied object par koi effect nahi hota.

Example:

```
import copy

original = [[1, 2], [3, 4]]

copied = copy.deepcopy(original)
original[0][0] = 99
print(original)
print(copied)
```

Output: [[99, 2], [3, 4]]

[[1, 2], [3, 4]]

ii. Shallow copy

Shallow Copy naya object create karti hai, lekin nested objects ka reference share karti hai. Is liye agar nested object me change karo to original aur copied dono me change nazar aata hai.

Example:

```
import copy

original = [[1, 2], [3, 4]]
copied = copy.copy(original)
original[0][0] = 99
print(original)
print(copied)
```

Output: [[99, 2], [3, 4]]
[[99, 2], [3, 4]]

18. Iterable vs Iterator ?

Answer: i. iterable

Iterable aik aisa object hota hai jis par hum iterate (loop) kar sakte hain. Ye `__iter__()` method provide karta hai aur is se iterator banaya ja sakta hai

Example:

```
numbers = [1, 2, 3]
for num in numbers:
    print(num) output: numbers iterable hai
```

Examples of Iterables i.List ii.Tuple iii.String iv.Dictionary v. Range

Note: Iterable = Collection of values → list, tuple, string, set, dict

ii. Iterator

Iterator wo object hota hai jo ek ek value return karta hai using `next()` function.

Example:

```
numbers = [1, 2, 3]
iterator = iter(numbers)
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Note: Iterator = One value at a time

19. What is a generator?

Answer: Generator aik special function hota hai jo `yield` keyword use karta hai aur values ko one by one generate karta hai

Generator sari values ko ek sath memory mein store nahi karta, Balke jab value ki zarurat hoti hai tab generate karke deta hai.

Example:

```
def gen():
```

```
    yield 10
    yield 20
g = gen()
print(next(g))
print(next(g))
```

Output: 10 20

21. yield aur return me kya difference hai?

Answer: `return` function ko completely khatam kar deta hai.

`yield` function ko pause karta hai aur agli call par wahi se continue karta hai.

22. Generator kab use karte hain?

Answer:

- Large files read karne ke liye
- Large datasets process karne ke liye
- Memory optimization ke liye
- Streaming data ke liye

20. Generator dobara use kyun nahi hota?

Answer: Generator ek one-time iterator hota hai. Jab uski tamam values consume ho jati hain to wo khatam ho jata hai. Is ke baad usay dobara use nahi kar sakte, naya generator create karna parta hai.

```
Example: def gen():
    yield 1
    yield 2
    yield 3
g = gen()
print(list(g))
print(list(g))
Output: [1, 2, 3]
[ ]
```

21. What is a decorator?

Answer: decorator aik function hai jo kisi dusre function ki functionality ko bina change kiy modify ya extend krta hai

Python me decorators @ syntax ke sath use hote hain.

Example: def my_decorator(func):

```
    def wrapper():
        print('start')
        func()
        print('end')
    return wrapper
```

```
@my_decorator
def my_func():
    print('hello world')
my_func()
```

output: start

hello world

end

Common uses of Decorators

- Logging
- Authentication
- Authorization
- Timing functions
- Caching
- Validation

21. What is oop?

Answer: OOP (Object-Oriented Programming) aik programming paradigm hai jisme hum real-world entities ko Classes aur Objects ki form me represent karte hain.

22. Class vs object?

Answer: i. Class aik blueprint (naqsha) hoti hai jiski basis par objects banaye jate

hain.

ii. Object class ka real instance hota hai jo memory me create hota hai.

25. instance aur Object mein difference ?

Answer: Practically dono terms ek hi meaning me use hoti hain
Har object class ka instance hota hai.

Examples: class Car:

```
    pass
    c1 = Car()
    c2 = Car()
    c3 = Car()
```

Output:

Class ka naam? car

Objects kitne? 3

Instances kitni? 3

26: Kya ek class se multiple objects bana sakte hain?

Answer: Haan, ek class se multiple objects ya instances create kiye ja sakte hai

Example: class Student:

```
    pass
    s1 = Student() ye
    s2 = Student() 3
    s3 = Student() objects hain
```

27. Attributes kitni Types k hote hain?

Answer: python may 2 types k attributes hote hain

i. **Class Attributes**

Class ke andar aur `__init__()` ke bahar define hota hai.
wo Class Attribute hote hain.

ii. **Instance Attributes**

`__init__()` ke andar **self** ke sath define hota hai.wo Instance Attribute hote hain.

Examples:

```
class Employee:
    company = "ABC Software" Class Attribute
    def __init__(self, name):
        self.name = name Instance Attribute
```

28. Python may methods ki types kitni hoti hain?

Answer: python may main 3 types k methods hote hain

i. instance method (self)

ii. Class method (@classmethod, cls)

iii. Static method (@staticmethod)

i. **Instance method**

instance method self parameter leta hai aur object (instance) ke data ke sath kaam karta hai.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
    def show(self): Note: yaha show instance method hai
        print(self.name)
s = Student("Nouman")
s.show()
```

ii. Class method

@classmethod decorator use karta hai aur cls parameter leta hai.
Ye class attributes ke sath kaam karta hai.

Example:

```
class Student:
    school = "Superior"
    @classmethod
    def show_school(cls): Note: yaha show_school class method hai
        print(cls.school)
Student.show_school()
```

iii. Static method

@staticmethod decorator use karta hai. Ye na object ke data se related hota hai aur na class ke data se.

Example: class Math:

```
    @staticmethod
    def add(a, b): Note: static method na self na na cls
        return a + b
print(Math.add(10, 20))
```

29. Self Kya Hota Hai?

Answer: self current object (instance) ka reference hota hai..

Example: class Student:

```
    def show(self):
        print("Hello")
s1 = Student()
s1.show()
```

30. What is constructor **__init__**?

Answer: Constructor **__init__()** Python ka special method hai jo

- **__new__()** → Object create karta hai
- **__init__()** → Object ko initialize karta hai
- **__init__()** object create hote hi automatically call hota hai.

31. What are Access Modifiers?

Answer: Access Modifiers class ke data aur methods ko kon access kar Sakta hai ye control karte hain

Python me 3 types ke Access Modifiers hote hain.

i.public

ii.protected

iii.private

i.Public members ko har jagah se access kiya ja sakta hai.

Example: class Student:

```
def __init__(self, name, age):  
    self.name = name Note: Public modifier  
    self.age = age Note: Public modifier
```

```
s = Student("Nouman", 25)  
print(s.name)  
print(s.age)
```

ii.protected members ko same class aur child class me use

karna chahiye.Ye `_` (single underscore) se start hote hain.

Example: class Student:

```
def __init__(self, name, age):  
    self._name = name Note: protected modifier  
    self._age = age Note: protected modifier
```

```
s = Student("Nouman", 25)  
print(s._name)  
print(s._age)
```

iii.Private members ko sirf class ke andar use karna chahiye.Ye `__` (double underscore) se start hote hain.

Example: class Student:

```
def show_name(self, name, age):  
    self.__name = name Note: private modifier  
    print(self.__name)  
s = Student()  
s.show_name("Nouman", 25)
```

32. Association kia hai?

Answer: Jab aik class ka object doosri class ke object se related ho, to usay Association kehte hain.

33. Kya Association me dono objects independently exist kar sakte hain?

Answer: Haan. Dono objects ek dusre ke bina bhi exist kar sakte hain.

Example: class Teachers:

```
def __init__(self, name):  
    self.name = name
```

class Students:

```
def __init__(self, name, teachers):  
    self.name = name  
    self.teachers = teachers
```

```
t1 = Teachers("Usama Bhai")  
s1 = Students("Nouman", t1)  
print(s1.teachers.name)
```

33. Aggregation kia hai:

Answer: Aggregation Association ki special type hai jahan ek class doosri class ka object rakhti hai lekin dono independently exist kar sakte hain.

Example: class Engine:

```
def __init__(self, name, engine_type):
    self.name = name
    self.engine_type = engine_type
class Train:
    def __init__(self, name, route, speed, engine):
        self.name = name
        self.route = route
        self.speed = speed
        self.engine = engine
engine = Engine("GUC 20", "Locomotive")

train = Train(
    "Ghouri Express",
    "Faisalabad to Lahore",
    "120 km/h",
    engine
)
print(train.name) output: Ghouri Express
print(train.engine.engine_type) output: Locomotive
print(train.engine.name) output: GUC 20
print(train.route) output: Faisalabad to Lahore
```

34. Aggregation aur Association me kya difference hai?

Answer: Association sirf relationship batati hai.

Aggregation me HAS-A relationship hoti hai aur ek class doosri class ka object rakhti hai.

34. Aggregation vs Composition

Answer: i. Aggregation

Aggregation aik weak HAS-A relationship hoti hai jahan Child object parent ke baghair bhi exist kar sakti hai.

ii. Composition

Composition aik strong HAS-A relationship hoti hai jahan Child object parent ke bina nahi kar sakta.

Example: class Engine:

```
def __init__(self):
    self.type = "V8"
class Car:
    def __init__(self):
        self.engine = Engine()
```

```
c1 = Car()
print(c1.engine.type) output: V8
```

24. Four Pillars of OOP?

Answer: i. Encapsulation

ii. Inheritance

iii. Polymorphism

iv. Abstraction

i. **Encapsulation** OOP ka aik concept hai jisme data (variables) aur methods (functions) ko aik class ke andar bundle kiya jata hai aur data ko direct access se protect kiya jata hai.

a) Why secure data and methods?

Data or methods ko secure is lia kia jata hai take unauthorized access na ho data safe rahe

b) What are Getter and Setter?

Getter or Setter Encapsulation k parts hain jo private data ko access or modify krne k lia use hote hain

i). **Getter** private variable ki value read or get karne k lia use hota hai

ii). **Setter** private variable ki value change ya update karne k lia use hota hai

Example: class Student:

```
def __init__(self, name, age):
    self.__name = name Note: Private
    self.__age = age Note: Private
```

Getter method

```
def get_name(self):
    return self.__name
```

Setter method

```
def set_name(self, name):
    self.__name = name
```

ii. **Inheritance** aik oop concept hai jis may child class parent class k methods or properties ko inherit krti hai

Type of inheritance

- Single Inheritance
- Multi-level inheritance
- Multiple inheritance
- Hierarchical Inheritance

a) **Single-Inheritance** Jb aik child class aik parent class se inherit kare

b) **Multi-level Inheritance** jb aik child class parent class se or parent class grandparent se inherit kare

c) **Multiple Inheritance** Job aik child class multiple parent classes se

inherit kare

d) **Hierarchical Inheritance** jb Multiple Child class aik parent class se Inherit kare

Examples:

i. Single Inheritance

```
class Animal:
    def eat(self):
        print("Eating")
class Dog(Animal):
    def eat(self):
        super().eat()
d1 = Dog()
d1.eat() output: Eating
```

ii. Multiple Inheritance

```
class Father:
    def skill1(self):
        print("Driving")
class Mother:
    def skill2(self):
        print("Cooking")
class Child(Father, Mother):
    def both(self):
        super().skill()
        super().skill()
c1 = Child()
c1.skill1() output: Driving
c1.skill2() output: cooking
```

iii. Multilevel Inheritance

```
class GrandParent:
    def house(self):
        print('old house')
class Parent(GrandParent):
    def house(self):
        return super().house()
class Child(Parent):
    def house(self):
        super().house()
c1 = Child()
c1.house() output: old house
```

Iv. Hierarchical Inheritance

```
class Animal:
    def eat(self):
        print("Eating")
class Dog(Animal):
    def def(self):
        super().eat()
```

```
class Cat(Animal):
    def eat(self):
        super().eat()
d = Dog()
c = Cat()
d.eat() output: Eating
c.eat() output: Eating
```

iii. Polymorphism kya hai?

Answer: Polymorphism aik OOP concept hai jisme same method ya function different objects ke liye different behavior provide karta hai.

Types of Polymorphism

1. Method Overloading
2. Method Overriding

i) What is Method Overloading?

Method Overloading me same class me same method different parameters ke sath use hota hai.

Important:

Python directly Method Overloading support nahi karti, lekin hum default arguments, *args aur **kwargs ki madad se overloading jaisa behavior achieve kar sakte hain.

ii) What is Method Overriding

Method Overriding me child class parent class ke method ko same name se dobara define karti hai aur apna behavior implement karti hai.

27. What is Abstraction?

Answer: Sirf important cheezain dikhana aur implementation details hide karna Abstraction kehlata hai.

Q1. Middleware kya hai

Middleware Django ka component hai jo Request aur Response ke darmiyan kaam karta hai aur request ko View tak aur response ko browser tak jane se pehle process karta hai.

i) What is Synchronous Programming

Synchronous Programming me code line by line execute hota hai. Ek task complete hone ke baad hi dusra task start hota hai. Isme program ka flow sequential (linear) hota hai.

ii) What is Asynchronous Programming?

Asynchronous Programming me jab aik task wait (I/O operation) me ho to program us waqt dusra task start kar sakta hai. Is se application zyada efficient hoti hai.

Note: Ye zaroori nahi ke tasks actual parallel chal rahe hon. Aksar ye

concurrently chalte hain (event loop ke through),

25. `super()` kia hai

Answer: `super()` parent class ke methods ya constructor ko child class se call karne ke liye use hota hai.

26. What is Method Overriding?

Answer: Jab child class parent class ke method ko same name se redefine kare.

27. Diamond problem kia hai?

Answer: Diamond Problem multiple inheritance mein tab hota hai jab ek class do aisi classes se inherit kare jo dono ek common parent class ko inherit karti hon, is se method resolution mein ambiguity (confusion) paida hota hai.

Example: class A:

```
def show(self):  
    print("A")
```

```
class B(A):  
    def show(self):  
        print("B")
```

```
class C(A):  
    def show(self):  
        print("C")
```

```
class D(B, C):  
    def show(self):  
        print("D")
```

```
print(D.mro())
```

27. MRO Kia hai?

Answer: MRO (Method Resolution Order) define karta hai k python pehle kis class k method ko call karega jab multiple classes may same method ho